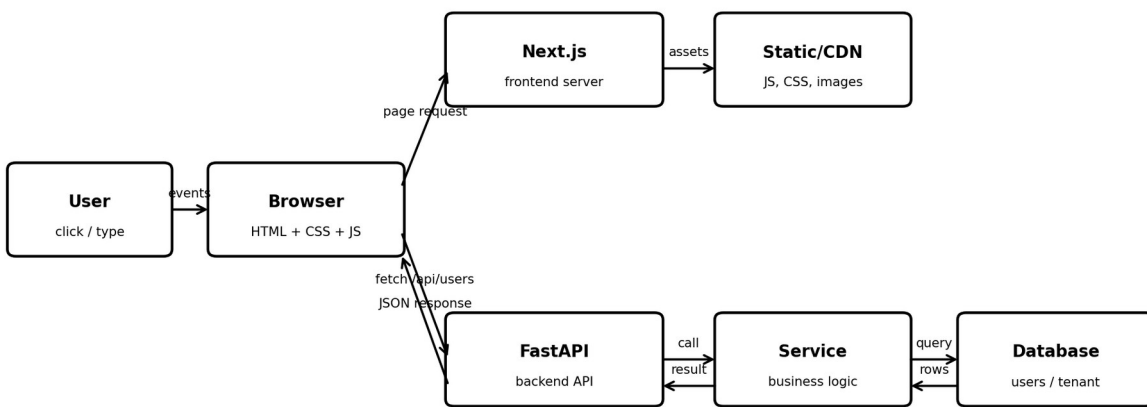


WEB SYSTEMS

MENTAL MODEL

How the Browser, Next.js, FastAPI, HTTP, Auth, CORS, DOM, and Configuration Actually Fit Together

The whole web app as a conversation



The guide follows this complete conversation from click to database and back.

A practical, intuition-first guide for developers who understand code logic but want to understand the web itself.

How to use this guide

The goal is not to memorize definitions. The goal is to build a mental simulator in your head. When you click a button, type a URL, see a 401, change an .env variable, or inspect a Network request, you should be able to narrate what is happening and predict what will happen if you change something.

The one question to ask everywhere: Who is sending what message, to which address, under which rules, and who is allowed to read or act on it?

Running example

Imagine an admin page at /settings/users. The page lists users belonging to the current tenant. The frontend is Next.js with TypeScript. The backend is FastAPI. The frontend calls GET /api/users?search=saad&page=1. The backend authenticates the caller, checks the tenant and permissions, queries data, and returns JSON. We will trace this same request repeatedly from different angles.

Contents

1. The web in one picture
2. What the browser really is
3. URLs, domains, origins, ports, DNS, TCP and TLS
4. HTTP request anatomy
5. HTTP response anatomy
6. What happens after you press Enter
7. Frontend: Next.js, React, components and routes
8. The DOM and rendering
9. Backend: FastAPI, routers, handlers, services and database
10. Middleware and dependencies
11. Frontend-backend communication
12. CORS and the same-origin policy
13. Cookies, sessions, tokens and JWT
14. Authentication vs authorization
15. Redirects and navigation
16. .env, configuration and modules
17. Errors and status codes
18. Browser DevTools: understand Inspect completely
19. What you can safely change vs structural plumbing
20. End-to-end request autopsy
21. Debugging playbook
22. Mini labs and exercises
23. Cheat sheets and glossary

1. The web in one picture

The whole web app as a conversation

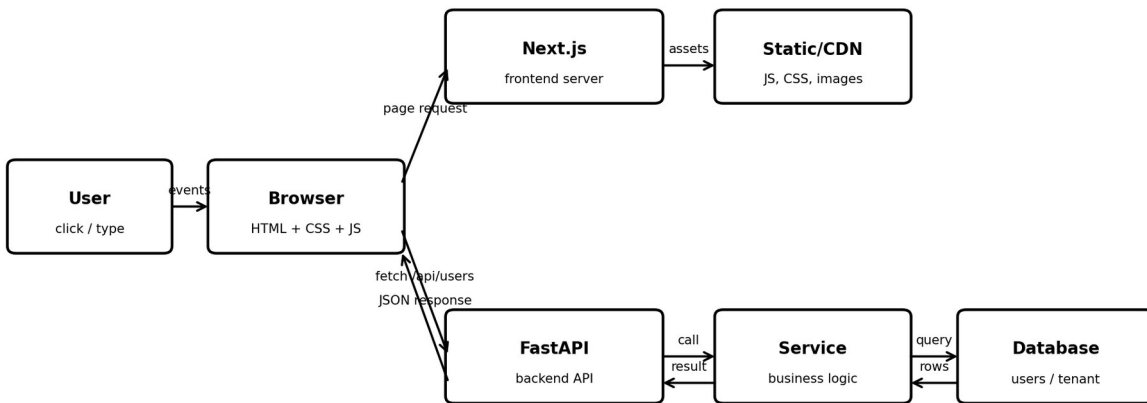


Figure 1 - A web application is a set of programs exchanging messages.

A modern web application is not one program. It is several programs running in different places. Your browser runs on your machine. A Next.js server may run locally or on a remote server. FastAPI runs as another server process. A database runs as another process or service. They cooperate by sending messages over defined interfaces.

Core mental model: A request is just a structured message. A response is another structured message. Frameworks make creating, routing, validating, and handling those messages easier.

The actors

Actor	Where it runs	Main responsibility
Browser	User device	Render UI, run JavaScript, send HTTP requests, enforce browser security rules.
Next.js	Browser + server, depending on component/code	Build pages, route URLs, render React, fetch data, serve assets.
FastAPI	Backend server	Expose API endpoints, validate requests, authenticate, run business logic, return responses.
Database	Database server/process	Persist and query data.
Reverse proxy/CDN	Edge/server	TLS termination, routing, caching, compression, static assets, load balancing.

A request is not a function call across machines

When frontend code calls `fetch('http://localhost:8000/users')`, JavaScript is not directly invoking a Python function. It asks the browser/runtime to send an HTTP message to an address. The backend server receives bytes, parses them as HTTP, matches the method and path, then eventually calls a Python function. This distinction explains many bugs: network failure, CORS, wrong host, wrong port, authentication, serialization, timeouts, and server errors can occur before your handler logic ever runs.

2. What the browser really is

The browser is much more than an IDE or a place that 'runs JavaScript'. It is a complete application runtime with a networking stack, HTML parser, CSS engine, JavaScript engine, DOM implementation, rendering engine, storage systems, security sandbox, developer tools, media systems, and an event loop.

Is the browser an IDE?

No. An IDE helps you write code. The browser executes web code and displays the result. DevTools can edit values temporarily, run JavaScript, inspect requests, and debug code, so it has IDE-like tools, but its primary job is to load resources, execute them under web security rules, and render a document.

Browser subsystem	What it does	What you see in DevTools
Networking	DNS, connections, HTTP, caching	Network tab
HTML parser	Converts HTML text into DOM nodes	Elements tab
CSS engine	Calculates which styles apply	Styles/Computed
JavaScript engine	Executes JS	Sources/Console
DOM APIs	Lets JS read/change page tree	Elements/Console
Storage	Cookies, localStorage, IndexedDB	Application tab
Renderer	Layout + paint + composite	Performance/Rendering tools
Security model	Same-origin, CORS, permissions	Console/Network/Security

JavaScript does not own the network

When JavaScript calls `fetch()`, the JS engine delegates networking to browser APIs. The browser performs the request asynchronously. When data arrives, the browser makes the result available to JavaScript through a Promise. That is why `async/await` is natural in web code: many operations depend on external events that finish later.

TypeScript / JavaScript

```
const response = await fetch('/api/users');
const data = await response.json();
console.log(data);
```

Think of `fetch()` as: "Browser, please send this HTTP request. Give me a Promise that becomes ready when a response is available."

3. URLs, domains, origins, ports, DNS, TCP and TLS

A URL is an address plus instructions

`https://app.example.com:443/api/users?page=2&search=ali#top`
|---| |-----| |---| |-----| |-----| |---|
scheme host port path query fragment

Part	Meaning	Example
Scheme	Protocol family / security	https
Host	Which server name	app.example.com
Port	Which network service on that host	443
Path	Which resource/route	/api/users
Query string	Extra request parameters	page=2&search=ali
Fragment	Client-side location; usually not sent to server	#top

Origin

For browser security, an origin is essentially scheme + host + port. Therefore `http://localhost:3000` and `http://localhost:8000` are different origins even though both are on your machine. This becomes important for CORS.

DNS: turning a name into an IP

Humans like `app.example.com`. Networks route using IP addresses. DNS maps a hostname to one or more IP addresses. After the browser resolves the hostname, it can connect to the target machine.

TCP/TLS: creating the pipe

HTTP needs a transport. Commonly, a connection is established to the server. With HTTPS, TLS authenticates the server certificate and encrypts traffic. After that, HTTP messages travel through the protected connection. Modern HTTP/2 and HTTP/3 optimize the transport, but your application-level mental model can still begin with request -> response.

Identity warning: The server seeing an IP address does not mean it knows which human is using the app. User identity normally comes from authentication credentials such as a session cookie or bearer token.

4. HTTP request anatomy

HTTP is structured messages carried by the network

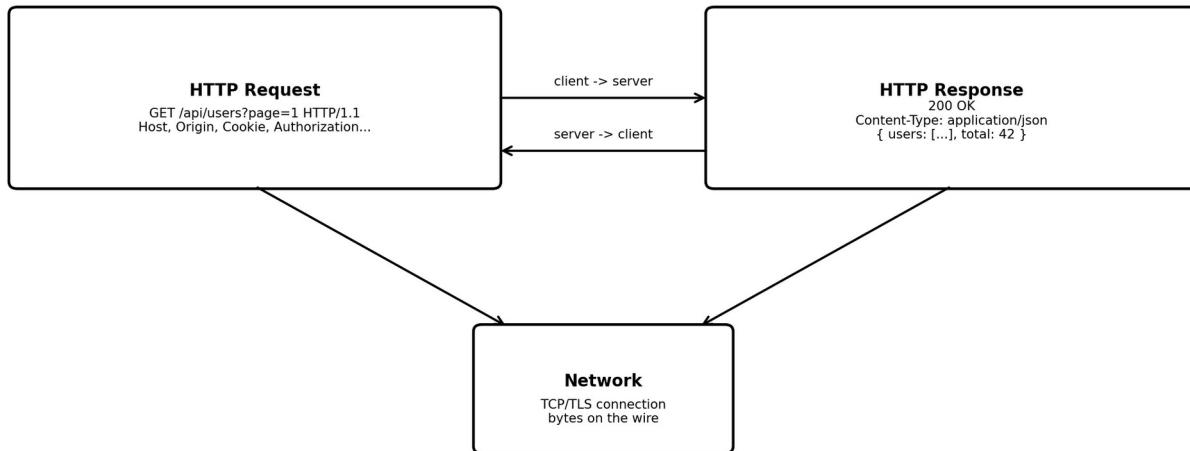


Figure 2 - HTTP messages have a start line, headers, and sometimes a body.

A typical request

```
HTTP
GET /api/users?page=1&search=saad HTTP/1.1
Host: api.example.com
Origin: https://app.example.com
Accept: application/json
Authorization: Bearer eyJ...
Cookie: session_id=abc123
User-Agent: Mozilla/5.0 ...
```

The request line says method + target path + HTTP version. Headers provide metadata. A body, if present, follows the headers. GET commonly has no body. POST, PUT, PATCH often carry JSON or form data.

Methods are intent labels

Method	Typical meaning	Example
GET	Read	GET /users
POST	Create / perform action	POST /users
PUT	Replace resource	PUT /users/42
PATCH	Partially update	PATCH /users/42
DELETE	Delete	DELETE /users/42
OPTIONS	Ask about communication options; used in CORS preflight	OPTIONS /users

Important request headers

Header	Why it exists
Host	Which hostname the request targets.
Content-Type	Format of the request body, e.g. application/json.

Header	Why it exists
Accept	Response formats the client can handle.
Authorization	Often carries bearer access tokens.
Cookie	Carries cookies matching the destination.
Origin	Tells the server which origin initiated certain browser requests.
Referer	May indicate the page that led to this request.
User-Agent	Information about client software; not reliable identity.

Body example

```

HTTP
POST /api/users
Content-Type: application/json

{
  "name": "Ali",
  "email": "ali@example.com"
}
```

FastAPI can parse this JSON body into a Pydantic model. That is not magic transport: the JSON bytes arrive first, then the framework decodes and validates them.

5. HTTP response anatomy

```

HTTP
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store
Set-Cookie: session_id=abc123; HttpOnly; Secure; SameSite=Lax

{
  "users": [
    {"id": 1, "name": "Ali"}
  ],
  "total": 1
}
```

The server chooses a status code, headers, and body. The browser does not need to know which Python function produced this. It only receives the HTTP response.

Who sends the response, and to whom?

Your endpoint function returns a Python value or Response object. FastAPI serializes it. The ASGI server (commonly Uvicorn) writes the HTTP response back on the connection associated with that request. The network stack delivers it to the client that opened the connection. You do not manually specify 'send this response to browser X'; the request/connection context already identifies where the response goes.

JSON is a contract

JSON is text representing data. On the backend, you may have Python dictionaries and Pydantic objects. On the frontend, response.json() parses the JSON into JavaScript values. The two languages do not share objects in memory; they serialize and deserialize data.

6. What happens after you press Enter

1. The browser parses the URL.
2. It checks caches, service workers, and security policies that may affect loading.
3. It resolves the hostname using DNS if needed.
4. It establishes a transport connection; HTTPS includes TLS.
5. It sends an HTTP request.
6. A web server/reverse proxy or application server receives the bytes.
7. The server framework parses method, path, headers, cookies, query and body.
8. Middleware can inspect or transform the request.
9. The router matches method + path to an endpoint.
10. Authentication/dependencies/business logic execute.
11. The backend may call a database or another service.
12. A response is built and sent back.
13. The browser checks status, headers, caching, CORS and content type.
14. For a page response, HTML is parsed into DOM; CSS and JS are loaded.
15. JavaScript may send additional API requests and update the UI.

Debugging consequence: A failure can happen at any layer. "The page is broken" is not one category of bug. Learn to locate the layer first.

7. Frontend: Next.js, React, components and routes

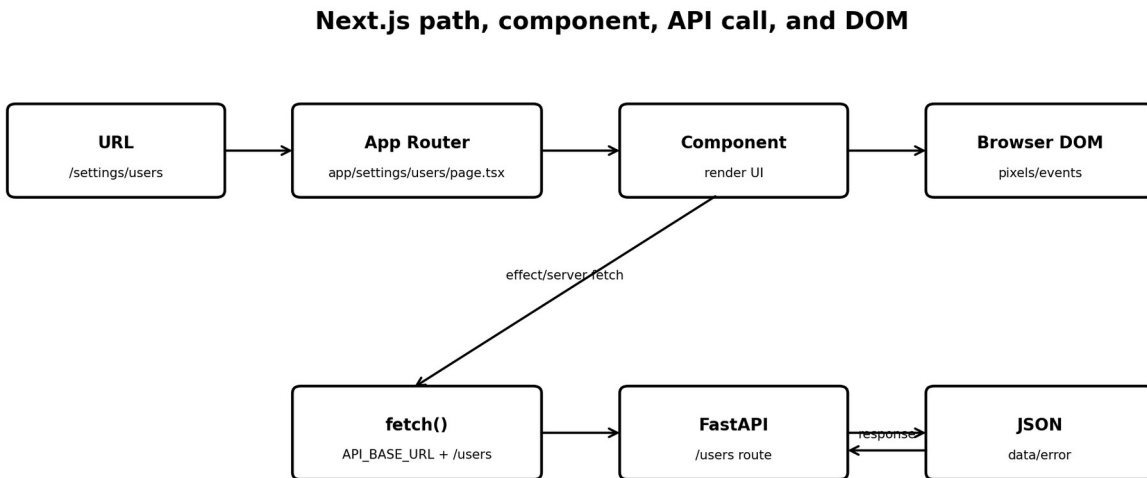


Figure 3 - URL route, page component, API call and rendered DOM are related but different.

What a React component is

A component is a function or module that describes a piece of UI. It receives data (props/state/context), returns a React element description, and can compose other components. It is not automatically a URL and not automatically an API endpoint.

```
TSX
type User = { id: string; name: string; email: string };

export function UserRow({ user }: { user: User }) {
  return (
    <tr>
      <td>{user.name}</td>
      <td>{user.email}</td>
    </tr>
  );
}
```

Next.js App Router: file path becomes URL path

```
Project tree
app/
  page.tsx          -> /
  settings/
    page.tsx        -> /settings
    users/
      page.tsx       -> /settings/users
      [id]/
        page.tsx     -> /settings/users/:id
```

page.tsx defines UI for a route segment. layout.tsx wraps nested routes. loading.tsx can define loading UI. error.tsx can define error boundaries. route.ts can define HTTP handlers inside Next.js when you intentionally want a server endpoint.

Client vs Server Components

In the App Router, components are Server Components by default. Add 'use client' when the component needs browser-only interactivity such as `useState`, `useEffect`, event handlers, or browser APIs. Server Components can fetch data on the server and send rendered results to the client; Client Components execute interactive logic in the browser.

```
TSX
'use client';

import { useEffect, useState } from 'react';

export default function UsersPage() {
  const [users, setUsers] = useState<User[]>([]);

  useEffect(() => {
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/users`)
      .then(r => r.json())
      .then(setUsers);
  }, []);

  return <UserTable users={users} />;
}
```

Important: `NEXT_PUBLIC_` variables can be exposed to browser code. Secrets must not be put in public frontend environment variables.

8. The DOM and rendering

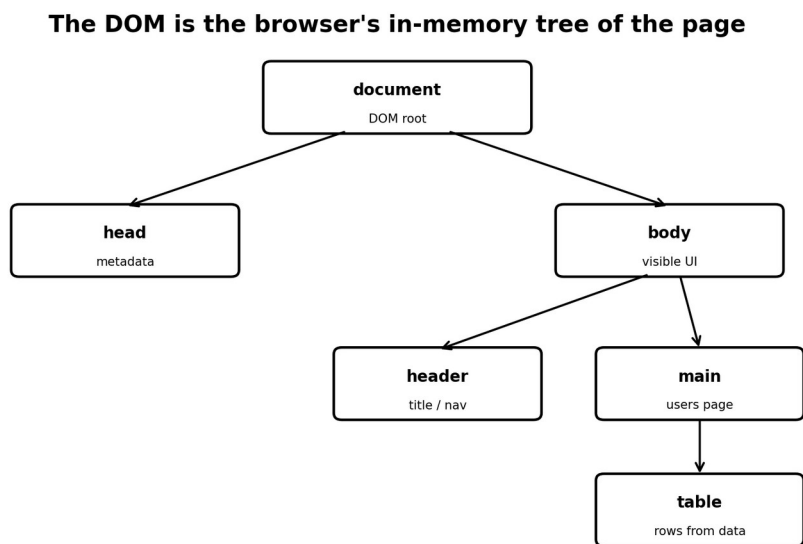


Figure 4 - The DOM is a tree of nodes representing the current document.

HTML is text. The DOM is the browser's in-memory object model created from that text and then changed over time. JavaScript can read and modify the DOM. React adds another abstraction: you describe desired UI, React reconciles changes, then updates the real DOM.

HTML vs DOM vs React tree

Thing	What it is	Where it exists
HTML	Text markup received/generated	File, network response, server output
React element tree	JavaScript descriptions of UI	React runtime
DOM	Browser objects such as document, div, table	Browser memory
Pixels	Final painted result	Screen

Why Inspect Element can differ from View Source

View Source shows original HTML response. The Elements panel shows the current DOM after JavaScript and React may have changed it. A table row fetched after page load may appear in Elements even if it was absent from the initial HTML.

Event flow

When you click a button, the browser produces an event. React's event handler may update state, call fetch, navigate, open a modal, or do nothing. The click itself does not automatically contact the backend. Only code that performs navigation, form submission, fetch/XHR, WebSocket activity, etc. creates network communication.

9. Backend: FastAPI, routers, handlers, services and database

Typical FastAPI request pipeline

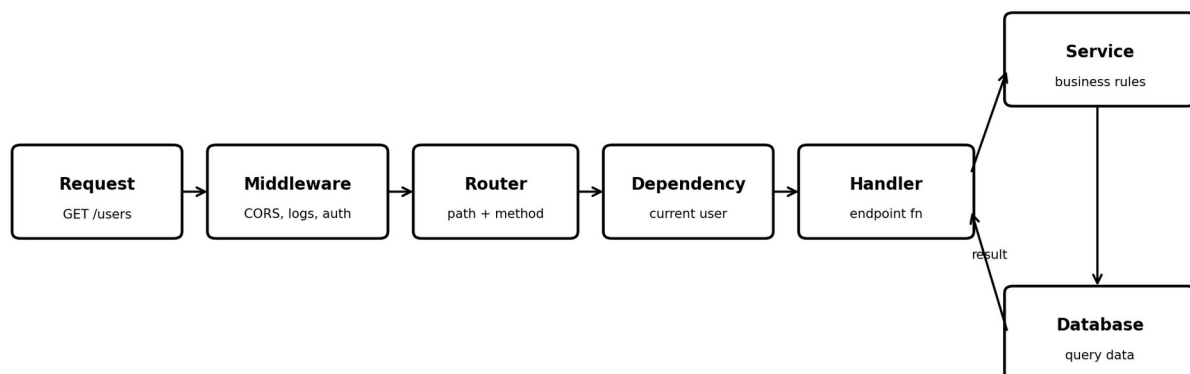


Figure 5 - A request usually passes through several backend layers.

Minimal FastAPI route

```
Python
from fastapi import FastAPI

app = FastAPI()
```

```
@app.get('/users')
def list_users():
    return {"users": []}
```

The decorator registers a route: when FastAPI receives a GET request whose path is /users, it can dispatch to list_users. The Python function name itself is not visible to the browser.

Routers organize endpoints

```
Python
# app/routers/users.py
from fastapi import APIRouter

router = APIRouter(prefix='/users', tags=['users'])

@router.get('/')
def list_users():
    ...

@router.get('/{user_id}')
def get_user(user_id: str):
    ...

# main.py
app.include_router(router, prefix='/api')
```

The final paths become /api/users and /api/users/{user_id}. Prefixes can be stacked. If you are confused about a path, build it from every prefix + the route decorator path.

Handler vs service vs repository

Layer	Typical responsibility	Should know HTTP?
Router/handler	Translate HTTP request into application call; choose response	Yes
Service	Business rules: permissions, workflows, calculations	Usually little/no
Repository/data access	Database queries	No HTTP needed
Model/schema	Data structure and validation	No HTTP needed

Projects vary. A demo may mix these layers. The important mental model is responsibility, not file naming. You can refactor when the codebase allows it, but first understand existing conventions.

10. Middleware and dependencies

Middleware wraps many or all requests. It is like a checkpoint around the router. It can log requests, attach correlation IDs, enforce security, add headers, handle CORS, measure timing, or catch errors.

```
Python
@app.middleware('http')
async def log_requests(request: Request, call_next):
    print(request.method, request.url.path)
    response = await call_next(request)
    print(response.status_code)
    return response
```

The onion model

```
request -> middleware A -> middleware B -> router -> handler
response <- middleware A <- middleware B <- router <- handler
```

A middleware can do work before and after the endpoint. That is why a request may fail before your route breakpoint is hit.

FastAPI dependencies

```
Python
from fastapi import Depends

async def current_user(...):
    # read cookie/token, validate it, load user
    return user

@router.get('/users')
async def list_users(user = Depends(current_user)):
    ...
```

Dependencies are explicit reusable prerequisites. Authentication, database sessions, tenant resolution, and parameter validation are common examples.

11. Frontend-backend communication

The frontend needs an address

```
TypeScript
const API_URL = process.env.NEXT_PUBLIC_API_URL;
const response = await fetch(`${API_URL}/api/users?page=${page}`);
```

The URL must resolve to wherever the backend is reachable from the code that performs the fetch. If the fetch runs in the browser, localhost means the user's machine. If it runs on a server container, localhost means that container itself. This single fact causes many environment bugs.

Absolute vs relative API paths

Style	Example	Meaning
Absolute	https://api.example.com/users	Directly call a specific origin.
Relative	/api/users	Call the same origin as the current page.
Proxied relative	/api/users -> reverse proxy -> FastAPI	Browser sees one origin while infrastructure forwards internally.

Data contract

```
TypeScript
// Frontend expectation
type UsersResponse = {
  items: User[];
  total: number;
  page: number;
};

Python
# Backend response schema
```

```
class UsersResponse(BaseModel):
    items: list[UserOut]
    total: int
    page: int
```

The frontend and backend are coupled through the API contract: path, method, query/body shape, auth expectations, status codes, and response shape. They do not need to be written in the same language.

12. CORS and the same-origin policy

CORS is a browser permission check between origins

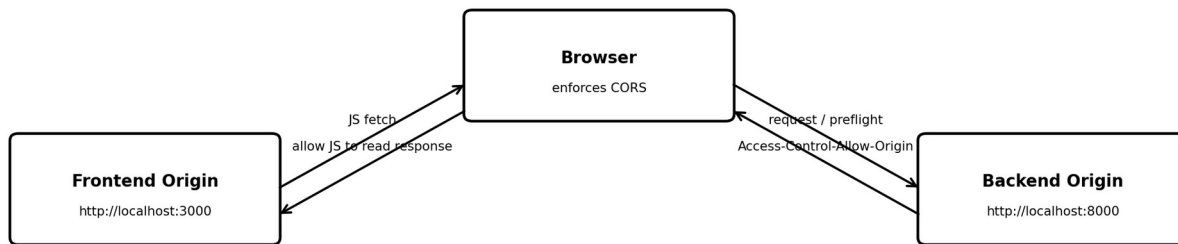


Figure 6 - CORS is primarily enforced by browsers, not by fetch itself or by FastAPI alone.

The same-origin policy prevents arbitrary websites from freely reading sensitive responses from other origins using your browser credentials. CORS is the controlled exception mechanism: the target server sends headers saying which origins, methods, headers, and credential modes are allowed.

Why Postman may work while browser fetch fails

Postman is not a normal web page running under browser same-origin rules. Your backend can be healthy and accessible while the browser refuses to expose the response to JavaScript because required CORS headers are missing.

FastAPI configuration

```
Python
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:3000'],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)
```

Preflight

For some cross-origin requests, the browser first sends an OPTIONS request asking whether the real request is permitted. This is called a preflight. If it fails, your POST/PATCH/etc. may never be sent.

Do not "fix" CORS by disabling browser security. Configure the allowed frontend origin correctly and understand whether credentials are involved.

13. Cookies, sessions, tokens and JWT

Authentication proves identity; authorization checks permission

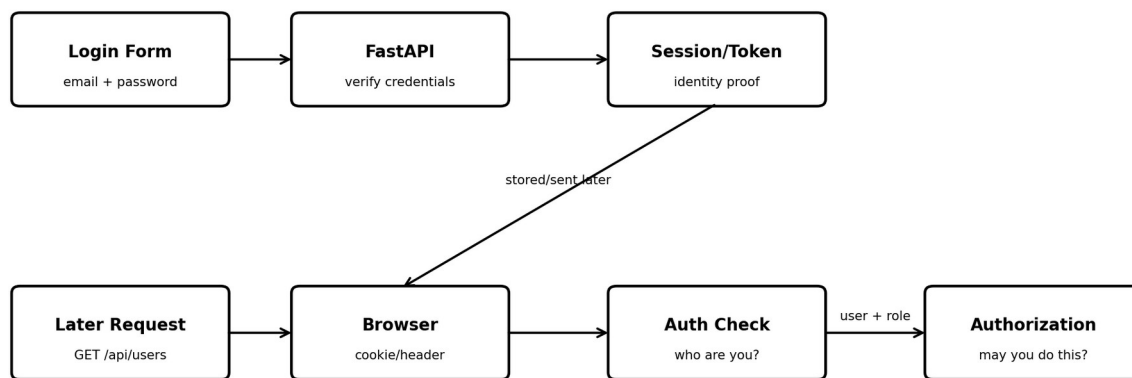


Figure 7 - Credentials established during login are reused on later requests.

Cookie

A cookie is a small name/value item stored by the browser under rules such as domain, path, expiry, Secure, HttpOnly and SameSite. When a request matches the cookie's rules, the browser may automatically attach it in the Cookie header.

Session

A session is a server-side concept: the server associates a session identifier with user state. The browser may store only a random session ID cookie. On each request the server looks up that ID and recovers the logged-in user.

Bearer token

```
HTTP
Authorization: Bearer <access-token>
```

A bearer token is a credential: whoever presents a valid token is treated as the bearer. Frontend code may attach it explicitly. Where it is stored matters for security.

JWT

JWT is a token format, not an authentication system by itself. It commonly contains signed claims such as subject/user ID, issuer, audience, expiry and roles. A server verifies the signature and claims before trusting it. JWT contents are usually encoded, not secret-encrypted by default.

Concept	Stored where?	Sent how?	Server state required?
Cookie	Browser cookie jar	Automatically in Cookie header when rules match	Cookie itself: no; session behind it: often yes
Session ID	Cookie or equivalent	Cookie/header	Yes, commonly
Access token	Memory/cookie/storage depending architecture	Authorization header or cookie	Not necessarily
JWT	Wherever token is stored	Usually Authorization header/cookie	Can be stateless, though real systems still keep other state

HttpOnly matters

An HttpOnly cookie cannot be read by normal JavaScript, which reduces exposure to token theft through XSS. But the browser can still attach it to matching HTTP requests.

14. Authentication vs authorization

Authentication answers: Who are you? Authorization answers: Are you allowed to perform this action on this resource? They are separate checks.

Tenant users example

1. Request contains session cookie or bearer token.
2. Backend validates the credential and identifies user 123.
3. Backend resolves that user's tenant, for example tenant A.
4. Backend checks role/permission: can this user list tenant users?
5. Backend constructs a database query scoped to tenant A.
6. Backend returns only allowed data.

Never trust tenant_id only because the frontend sent it. If tenant boundaries are security-sensitive, derive or validate tenant scope on the backend from authenticated identity and permissions.

Common status distinction

Situation	Typical status
No valid identity / credentials	401 Unauthorized
Identity known but action forbidden	403 Forbidden

Situation	Typical status
Resource does not exist or intentionally hidden	404 Not Found may be used
Valid request and allowed	2xx

15. Redirects and navigation

Two very different redirects

Type	Who initiates it?	What happens
HTTP redirect	Server response, e.g. 302/307 + Location header	Browser/client follows another URL depending rules.
Client navigation	JavaScript/router, e.g. router.push("/login")	Frontend changes route without a traditional full-page redirect.

Server redirect

```
HTTP/1.1 302 Found
Location: /login
```

Next.js client navigation

```
'use client';
import { useRouter } from 'next/navigation';

const router = useRouter();
router.push('/login');
```

In DevTools Network, an HTTP redirect appears as a response status followed by another request. Client navigation may trigger framework-specific requests rather than a classic 302.

16. .env, configuration and modules

Why configuration exists

Code describes behavior. Configuration supplies values that differ by environment: API URLs, database URLs, feature flags, logging levels, secrets, OAuth client IDs, etc. Keeping them outside source logic avoids hardcoding deployment-specific values.

```
# .env.local (frontend)
NEXT_PUBLIC_API_URL=http://localhost:8000

# backend .env
DATABASE_URL=postgresql://...
JWT_SECRET=...
ALLOWED_ORIGINS=http://localhost:3000
```

Public vs secret

Value	Can browser know it?	Where it should live
Public API base URL	Usually yes	Public frontend config can be fine
Database password	No	Backend secret environment
JWT signing secret	No	Backend secret environment
Public OAuth client ID	Often yes	Frontend/public config
Private API key	No	Backend secret environment

Environment variables are strings

A process receives environment variables from its operating environment. Frameworks may load .env files during development. The .env file is a convenience input, not a magical runtime database. Values often need parsing into booleans, integers, lists, or URLs.

Config modules

```
Python
# config.py
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file='.env')

    database_url: str
    allowed_origins: list[str] = []

settings = Settings()
```

A config module centralizes parsing and validation so the rest of your app imports settings instead of reading environment variables everywhere.

Modules

A module is simply a unit of code that can export/import names. In JavaScript/TypeScript, files commonly act as ES modules. In Python, .py files and packages are modules. Modules help organize responsibility; they are not network concepts.

17. Errors and status codes

Status family	Meaning	Examples
1xx	Informational	rarely handled directly in app code
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection/cache	301, 302, 304, 307, 308
4xx	Client-side/request problem	400, 401, 403, 404, 409, 422, 429
5xx	Server failed to fulfill valid request	500, 502, 503, 504

Transport error vs HTTP error

fetch() can fail because no response was obtained at all: DNS failure, refused connection, TLS error, network interruption. Separately, the server can return an HTTP error such as 404 or 500. fetch() normally resolves for HTTP 404/500; you must inspect response.ok or response.status.

```
TypeScript
const response = await fetch(url);
if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
}
const data = await response.json();
```

422 in FastAPI

FastAPI commonly returns 422 when request data does not satisfy declared validation. Before debugging database logic, inspect the request path/query/body and the validation error details.

18. Browser DevTools: understand Inspect completely

Network tab: your main truth source

Field/tab	Question it answers
Name / URL	What address was requested?
Method	GET/POST/etc.?
Status	What did the server return?
Initiator	What code/navigation triggered it?
Request Headers	What metadata/credentials/origin were sent?
Query String	What URL parameters were sent?
Request Payload	What body was sent?
Response Headers	What rules/content metadata came back?
Response/Preview	What body came back?
Timing	Where was time spent?
Cookies	Which cookies were sent/set?

A systematic Network inspection

1. Filter by Fetch/XHR if you are debugging API calls.
2. Click the failing request.
3. Read Request URL and Request Method. Confirm host, port and path.
4. Read Status Code.
5. Inspect Query String Parameters or Request Payload.
6. Inspect Authorization and Cookie headers when relevant.
7. Inspect Origin and CORS response headers for cross-origin issues.
8. Read the Response body; many backends tell you exactly what failed.
9. Use Initiator to find the frontend code that triggered the request.
10. Check Timing only after correctness is established.

Elements tab

Use Elements to inspect the current DOM and computed CSS. Editing here is temporary: it changes the browser's current page state, not your source files.

Console

Use Console for runtime JavaScript errors, logs, CORS messages, rejected promises, and experimenting with DOM/JS expressions. A red Console message may be a symptom of a failed network request rather than the root cause.

Application tab

Use Application to inspect cookies, localStorage, sessionStorage, IndexedDB, service workers and caches. This is often where login/session debugging becomes concrete.

Sources

Use Sources to set breakpoints and inspect browser-executed code. With source maps, you can often debug your original TypeScript/React source rather than bundled JavaScript.

Debugging superpower: Before changing code, prove which request failed and what exact message crossed the boundary.

19. What you can safely change vs structural plumbing

A major step toward coding freely is separating product/UI choices from contracts and infrastructure. Nothing is absolutely unchangeable, but some changes are local while others require coordinated changes across layers.

Thing	Usually safe to change locally?	Why / what to check
Text, spacing, visual layout in a component	Yes	Check design system/tests/accessibility.
Component internal state implementation	Often	Keep external props/behavior contract.
API response field name	No, not alone	Frontend/backend contract must change together.
Backend route path	No, not alone	Every caller/proxy/test must use new path.
Environment variable name	No, not alone	Deployment config + code + docs must match.
Authentication middleware	High risk	Affects security and many routes.
CORS allowed origin	Config change	Must match actual frontend origin and credential mode.
Database schema	Coordinated	Migrations + backend queries + possibly API contract.
CSS class	Usually	Unless tests/design system/JS selectors rely on it.
React component file name	Usually	Update imports; route-special files have framework meaning.
page.tsx location	Structural	Its folder defines URL route.
FastAPI decorator method/path	Structural contract	Defines what HTTP request

Thing	Usually safe to change locally?	Why / what to check
		matches the handler.

Ask these four questions before editing

1. Is this value visible outside this module/component?
2. Does another layer depend on its name, path, shape or side effect?
3. Is the framework giving this file/name special meaning?
4. Do tests, configuration, deployment or security rules depend on it?

Framework convention vs ordinary code

A button component is ordinary application code: reshape it as long as its users still get what they expect. `app/users/page.tsx` is partly application code and partly framework convention: moving it changes routing. `middleware.ts`, `next.config.*`, FastAPI app creation, CORS setup and dependency wiring can affect broad behavior. Learn to recognize these boundaries, not fear them.

20. End-to-end request autopsy: tenant users

Scenario

An authenticated admin visits `https://app.example.com/settings/users` and searches for 'saad'. The frontend should display page 1 of matching users from the same tenant.

Step A - Navigation

The browser requests or navigates to `/settings/users`. Next.js maps that URL to the matching App Router segment, such as `app/settings/users/page.tsx`. Depending on rendering mode, Next.js may render server-side content, send React payloads, or hydrate client code.

Step B - Component decides data is needed

```
TypeScript
const params = new URLSearchParams({
  search,
  page: String(page),
  limit: '20',
});

const res = await fetch(`${API_URL}/api/users?${params}`, {
  credentials: 'include',
});
```

If this executes in the browser, the browser owns the HTTP request. `credentials: 'include'` matters when cross-origin cookies must be sent and the server's CORS policy allows credentials.

Step C - Request on the wire

```
HTTP
GET /api/users?search=saad&page=1&limit=20 HTTP/1.1
Host: api.example.com
Origin: https://app.example.com
```

```
Cookie: session_id=...
Accept: application/json
```

Step D - Backend entry

Uvicorn receives the request and hands it to the FastAPI application through ASGI. Middleware may add logging and CORS handling. The router matches GET /api/users.

Step E - Authentication and tenant context

A dependency reads the session/token, validates it, loads the user, resolves tenant context, and perhaps checks an admin permission. If authentication fails, the endpoint may never execute its main body.

Step F - Validation

```
Python
@router.get('/users')
async def list_users(
    search: str | None = None,
    page: int = 1,
    limit: int = 20,
    current = Depends(require_admin),
):
    ...
```

FastAPI converts page and limit from URL strings into integers and validates constraints if declared.

Step G - Query

```
Python
users = await repo.list_users(
    tenant_id=current.tenant_id,
    search=search,
    offset=(page - 1) * limit,
    limit=limit,
)
```

The backend, not the UI, should enforce the tenant boundary. Search, filters and pagination become query conditions.

Step H - Response

```
JSON
{
  "items": [
    {"id": "u_17", "name": "Saad", "email": "..."}
  ],
  "total": 7,
  "page": 1,
  "limit": 20
}
```

Step I - Browser receives and React rerenders

The fetch Promise resolves. Frontend code parses JSON, stores it in state or a data cache, and React renders rows. React updates the DOM. The browser recalculates layout/paint as needed and you see the table.

Where every common bug would appear

Symptom	Likely layer to inspect first
ERR_CONNECTION_REFUSED	Backend not listening / wrong host/port / container networking
CORS error	Origin/CORS middleware/preflight/credentials
401	Cookie/token missing/expired/invalid
403	Authenticated but permission denied
404	Wrong path/prefix/method or proxy routing
422	Query/body shape validation mismatch
500	Backend exception
200 but empty table	Response shape/state/rendering/filter logic
Correct API response but stale UI	React state/cache/revalidation problem
Works server-side, fails browser-side	CORS, browser cookies, public URL, client environment

21. Debugging playbook

When "frontend cannot get data"

1. Open Network. Is a request sent at all?
2. If no: inspect event handler/effect/server component logic and Console.
3. If yes: verify exact URL, method, query and payload.
4. If connection fails: verify backend process, address, port and container/proxy networking.
5. If CORS: inspect Origin, preflight OPTIONS, allow-origin and credentials settings.
6. If 401/403: inspect cookie/Authorization header and backend auth logs.
7. If 404: reconstruct route prefix + router path + reverse proxy path.
8. If 422: compare payload/query names and types with FastAPI schema.
9. If 500: inspect backend traceback and request correlation logs.
10. If 2xx: inspect response JSON and then frontend parsing/state/rendering.

Trace IDs are powerful

In larger systems, middleware can assign a request ID. Put that ID in logs and response headers. Then one browser request can be correlated with backend logs and downstream calls.

Read from the boundary inward

Best order: Network message -> routing -> authentication/validation -> business logic -> database -> response -> frontend state -> DOM.

This is usually faster than staring at source code and guessing. The network boundary gives observable facts.

22. Mini labs and exercises

Lab 1 - See the request

1. Run Next.js and FastAPI locally.
2. Open /settings/users.
3. Open DevTools -> Network -> Fetch/XHR.
4. Trigger a search.
5. Write down Request URL, Method, Status, Query String, Request Headers, Response Headers and Response body.
6. Find the Initiator and locate the frontend source line.

Lab 2 - Break the route on purpose

Change /api/users to /api/userzzz only in the frontend. Predict the status before refreshing. Restore it. Then change the FastAPI prefix and observe the same contract break from the opposite side.

Lab 3 - CORS experimentally

Temporarily remove http://localhost:3000 from allowed origins. Observe the browser error and the Network entries. Then call the endpoint from curl and compare. Restore the correct configuration immediately.

Lab 4 - Authentication

Inspect Application -> Cookies. Identify the cookie used by your own development app, if it uses cookie sessions. Log out and compare the request headers before and after. If your app uses bearer tokens instead, inspect the Authorization header without copying real secrets into notes or chats.

Lab 5 - DOM vs source

Open View Source, then Elements. Find a user row loaded asynchronously. Explain why it can exist in Elements but not in initial source.

Lab 6 - Build a tiny endpoint

```
Python
# backend
@router.get('/debug/echo')
async def echo(request: Request, q: str | None = None):
    return {
        'method': request.method,
        'path': request.url.path,
        'query': q,
        'origin': request.headers.get('origin'),
    }
```

```
TypeScript  
// frontend  
const r = await fetch(`${API_URL}/debug/echo?q=hello`);  
console.log(await r.json());
```

Use this only in local development. The purpose is to see the transformation: JavaScript values -> HTTP -> FastAPI request object -> JSON -> JavaScript values.

Challenge questions

1. Why can GET `http://localhost:8000/users` work in curl but fail from a page served at `http://localhost:3000`?
2. If FastAPI receives `/api/users` but your decorator is `@router.get("/users")` and the router prefix is `/api`, will it match? Explain.
3. Why is `process.env.NEXT_PUBLIC_API_URL` not a safe place for a database password?
4. What is the difference between a 401 response and a network failure where no HTTP response exists?
5. Why can a button click happen with zero network requests?
6. If a response is 200 and contains correct users but the table is empty, which layers are already proven to work?
7. Why might localhost inside a Docker container not refer to your host machine?
8. What exactly makes `http://localhost:3000` and `http://localhost:8000` different origins?
9. What is the difference between authentication and tenant authorization?
10. When DevTools says an OPTIONS request failed, why might your POST never reach its route handler?

Answers

1. Because browsers enforce same-origin/CORS rules for page JavaScript; curl does not use the browser same-origin policy.
2. Yes. `/api` prefix + `/users` route becomes `/api/users`.
3. `NEXT_PUBLIC_` values can be bundled/exposed to client-side browser code. A database password must stay server-side.
4. 401 is a valid HTTP response from a reachable server. A network failure means the client could not obtain an HTTP response.
5. A click is only an event. It creates network traffic only if its handler/navigation/form behavior triggers it.
6. Network transport, route, backend logic and response are likely functioning; inspect parsing, state/cache and rendering.
7. Each container has its own network namespace. localhost refers to the current container unless networking is configured otherwise.

8. The ports differ; origin includes scheme + host + port.

9. Authentication proves the caller identity; tenant authorization determines whether that identity may access that tenant/resource.

10. The browser uses the preflight result as permission to send the actual cross-origin request. If permission fails, it blocks the real request.

23. Cheat sheets and glossary

Request mental checklist

Question	Where to inspect
Who initiated it?	Network -> Initiator / frontend source
Where is it going?	Request URL
What action?	Method
What data?	Query / Payload
Who does server think I am?	Cookie / Authorization + backend auth
Why blocked?	Status / Console / CORS headers
What came back?	Response / Preview
Which backend function?	Method + full path -> router registration

FastAPI route equation

Final endpoint path = app-level prefix + included-router prefix + APIRouter prefix + decorator path (depending on how your project composes routers).

Next.js App Router equation

Page URL = folder segments under app/ that participate in routing. Special files such as page.tsx and layout.tsx provide behavior for those segments.

Glossary

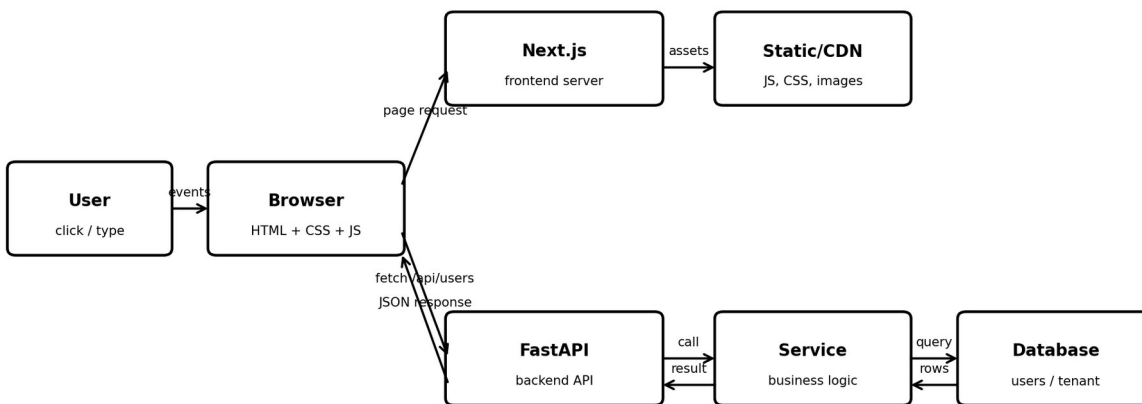
Term	Mental definition
API	A defined interface through which software communicates.
Endpoint	A specific method + path combination exposed by a server.
Route	A rule mapping a path/method to behavior or UI.
Client	Program initiating a request.
Server	Program listening for and responding to requests.
Origin	Scheme + host + port used by browser security rules.
Header	Metadata attached to an HTTP request/response.
Payload/body	Main data carried by a request or response.
Cookie	Browser-managed small data item sent according to cookie rules.
Session	Server-side continuity/identity state linked across requests.
Token	Credential or encoded proof used to authorize requests.
CORS	Browser cross-origin permission mechanism controlled by

Term	Mental definition
	response headers.
Middleware	Code wrapping request/response processing across routes.
Router	Dispatch table mapping request patterns to handlers.
Handler/endpoint function	Function that handles a matched request.
DOM	Browser object tree representing the current document.
Hydration	Attaching client React behavior to server-rendered UI.
Serialization	Converting in-memory data to transferable format such as JSON.
Deserialization	Parsing transferred data back into language values.
Environment variable	External string configuration provided to a process.
Reverse proxy	Server that receives requests and forwards them to internal services.
TLS	Protocol providing encryption and server authentication for HTTPS.
DNS	System resolving hostnames to IP addresses.
Status code	Numeric HTTP result category.
Preflight	CORS OPTIONS request used to check permission before certain requests.

The mental model to keep

When you work on a web project, do not think only in files. Think in boundaries and messages. A component renders UI. An event triggers code. Code may ask the browser to send a request. The request has an address, method, headers and data. Infrastructure delivers it. Middleware and routers decide where it goes. Authentication establishes identity. Authorization checks permission. Business logic queries data. The server serializes a response. The browser receives it. JavaScript turns it into state. React turns state into a UI description. The DOM and rendering engine turn that into pixels.

The whole web app as a conversation



Final map - narrate this flow whenever you are unsure where a bug or change belongs.

Freedom comes from contracts. Once you know which parts are local implementation details and which parts are cross-layer contracts, you can change code confidently instead of being afraid of the framework.

A final debugging mantra

1. Observe the message.
2. Verify the address and contract.
3. Find the layer that rejected or transformed it.
4. Change one layer intentionally.
5. Observe the message again.

If you can do those five steps, you are no longer treating the web as magic. You are operating a system you can inspect, reason about, and modify.